

Spec 07 — AI Search Module

Tell Copilot: "**implement spec 07**"

Depends on: spec 06 (pipeline).

Read `specs/business-rules.md` R3, R6–R11, R15 before starting.

Also load `.github/instructions/talentlens-ai-search.instructions.md` — it governs every file in this module.

What this spec builds

`server/src/search/` — three services (orchestration, Qwen 3.5 ranking, heuristic fallback) + controller + DTOs.

No embedding pipeline. The manager's query goes directly into the Qwen 3.5 prompt as the primary signal.

Endpoints

Method	Path	Auth	Description
POST	<code>/search</code>	manager, hr	AI-ranked candidate matches for an IRC
POST	<code>/search/upload-jd</code>	manager, hr	Upload PDF/DOCX → extract text → run search

SearchDto (`search.dto.ts`)

```
ircId:    number    // @IsInt – must be an Open IRC (R2)
query?:   string     // @IsOptional @IsString – manager's free-text
           requirement (PRIMARY signal)
scope:    'all' | 'applied' // @IsIn(['all','applied'])
jdText?:  string     // @IsOptional @IsString – extracted JD text
           (SECONDARY context)
```

SearchResultDto (shape returned per matched employee)

```
employeeId:    number    // from model
fullName:      string     // re-hydrated from DB
roleTitle:     string     // re-hydrated from DB
location:      string     // re-hydrated from DB
businessUnit:  string     // re-hydrated from DB
experienceYears: number   // re-hydrated from DB
skills:        string[]  // re-hydrated from DB
currentAllocation: string | null
```

```

availableDate:      string | null
matchPct:           number // from model (0-100)
whyRecommend:       string // from model – must cite project evidence
(R6)
whyNot:             string[] // from model – always at least 1 item
(R7)
conflict:           boolean // from model
conflictNote:       string | null
isDuplicate:        boolean // from DB check (R5)
duplicateNote:      string | null
alreadyInPipeline:  boolean // from DB check (R4)
pipelineCandidateId: number | null

```

Three services — responsibilities are fixed

search.service.ts — orchestration only

1. Load IRC + parent project from Prisma. Throw `BadRequestException` if IRC is not `Open` (R2).
2. Build candidate pool:
 - `scope=applied`: employees in this IRC's active pipeline only (R10).
 - `scope=all`: pre-filter by mandatory-skill overlap; cap at `MAX_RANKING_CANDIDATES` (R15 excludes Rejected).
3. Call `LlamaService.rank(irc, pool, query, jdText)`.
 - On any failure → call `HeuristicService.rank(irc, pool)` silently.
4. Re-hydrate every result from DB — ALL display fields replaced.
5. Duplicate-check (R5) and `alreadyInPipeline` check (R4).
6. Log to `search_logs`.
7. Return sorted by `matchPct` desc (R9).

llama.service.ts — prompt + API + Zod only

1. Build `effectiveQuery` from query + optional `jdText` (see instructions for priority rules).
2. Construct the prompt with manager query **first**, IRC context **second** (see instructions).
3. `POST ${LLAMA_BASE_URL}/api/generate` with `{ model, prompt, stream: false, options: { temperature: 0 } }`.
4. Parse `response` field and validate with Zod schema.
5. Retry once on parse failure, then throw typed error.

heuristic.service.ts — synchronous fallback, no I/O

```

matchPct = (matched mandatory / total mandatory) × 100, capped at 85
whyRecommend = "Matches {n} of {total} mandatory skills: {list}"
whyNot      = unmatched mandatory skills (at least 1 – R7)
conflict    = availableDate > project.startDate

```

JD upload (POST /search/upload-jd)

- Accept `multipart/form-data` with field `file` (PDF or DOCX) and `ircId`, `scope`, optional `query`.
 - PDF → `pdf-parse`, DOCX → `mammoth`. Throw `BadRequestException` for other types (R11).
 - If extraction yields empty text: fall back to `query` alone, log warning (do not fail request).
 - Pass extracted text as `jdText` into the same `SearchService.search()` flow.
 - Multer: `memoryStorage()`, PDF/DOCX filter, `fileSize: 5MB`.
-

Constants

```
// server/src/common/constants/search.constants.ts
export const MAX_RANKING_CANDIDATES = 35;
export const JD_TEXT_MAX_LENGTH    = 4000;
```

Config must expose and validate:

- `LLAMA_BASE_URL`, `LLAMA_API_KEY` (optional), `LLAMA_MODEL`, `RANKING_PROVIDER`
-

Acceptance criteria

- `POST /search { ircId:1, query:"engineers who worked on payments", scope:"all" }` → Qwen-ranked array with `whyRecommend` citing project history.
- Satya Mishra appears with `alreadyInPipeline: true`.
- Invalid/missing Ollama config → heuristic fallback, no 500 error.
- Closed IRC → `400 Bad Request`.
- `POST /search/upload-jd` with PDF → extracted text used as secondary JD context.
- `scope:applied` returns only candidates already in this IRC's pipeline.
- Every search writes a `search_logs` row.